LAB 5 – STATIC CODE ANALYSIS

NAME : PRABHAKAR KUMAR

SRN : PES2UG24CS355

SEC : F

GITHUB QUESTION LINK : https://github.com/SETAPESU26/L5_47_ecommerce_cart_system?utm_source=chatgpt.com

PROJECT: E-commerce Cart System

Ecommerce_cart_system.py (Before)

```
PS C:\Users\Prabhakar\L5_47_ecommerce_cart_system> python ecommerce_cart_system.py
Is ITEM1 in cart: True
Discounted Price: 22.5
Item Hash: 86a07ea6926052ccd56f9bc3cdcc10ef
Checkout PIN: 7661
Receipt saved to: receipt.txt
Total items: 6
Gateway secret: STRIPE_SECRET_KEY_LIVE_2026
```

After :

```
(venv) PS C:\Users\Prabhakar\L5_47_ecommerce_cart_system> python ecommerce_cart_system.py
INFO: Added item ITEM4 to the cart.
INFO: Removed item ITEM2 from the cart.
Is ITEM1 in cart: True
Discounted Price: 22.5
Item Hash: de124cf18e4be214c25c0c75410474d1b4c4b45679baa47caf3404bd34c806dc
Checkout PIN generated securely.
Total items: 6
Cart total: $220.00
INFO: Audit: Added ITEM4 at 35.00 each.
INFO: Receipt saved to receipt.txt
```

STATIC ANALYSIS RESULT :

1). pylint_report_before.txt

```
(venv) PS C:\Users\Prabhakar\L5_47_ecommerce_cart_system> Get-Content pylint_report.txt
************* Module ecommerce_cart_system
ecommerce_cart_system.py:84:0: C0304: Final newline missing (missing-final-newline)
ecommerce_cart_system.py:1:0: C0114: Missing module docstring (missing-module-docstring)
ecommerce_cart_system.py:12:0: C0103: Constant name "total_sales" doesn't conform to UPPER_CASE naming style (invalid-name)
ecommerce_cart_system.py:15:0: C0116: Missing function or method docstring (missing-function-docstring)
ecommerce_cart_system.py:15:0: C0103: Function name "addItemToCart" doesn't conform to snake_case naming style (invalid-name)
ecommerce_cart_system.py:15:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
ecommerce_cart_system.py:16:4: W0603: Using the global statement (global-statement)
ecommerce_cart_system.py:21:21: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
ecommerce_cart_system.py:25:0: C0116: Missing function or method docstring (missing-function-docstring)
ecommerce_cart_system.py:25:0: C0103: Function name "removeItemFromCart" doesn't conform to snake_case naming style (invalid-name)
ecommerce_cart_system.py:28:4: W0702: No exception type(s) specified (bare-except)
ecommerce_cart_system.py:32:0: C0116: Missing function or method docstring (missing-function-docstring)
ecommerce_cart_system.py:32:0: C0103: Function name "checkCartStatus" doesn't conform to snake_case naming style (invalid-name)
ecommerce_cart_system.py:34:7: C0121: Comparison 'cart_items.get(item_id) == None' should be 'cart_items.get(item_id) is None' (singleton-comparison)
ecommerce_cart_system.py:36:4: R1703: The if statement can be replaced with 'return bool(test)' (simplifiable-if-statement)
ecommerce_cart_system.py:36:4: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside it (no-else-return)
ecommerce_cart_system.py:33:4: W0612: Unused variable 'handling_fee' (unused-variable)
ecommerce_cart_system.py:42:0: C0116: Missing function or method docstring (missing-function-docstring)
ecommerce_cart_system.py:42:0: C0103: Function name "calculateDiscountedTotal" doesn't conform to snake_case naming style (invalid-name)
ecommerce_cart_system.py:44:19: W0123: Use of eval (eval-used)
ecommerce_cart_system.py:43:4: W0612: Unused variable 'base_price' (unused-variable)
ecommerce_cart_system.py:48:0: C0116: Missing function or method docstring (missing-function-docstring)
ecommerce_cart_system.py:48:0: C0103: Function name "generateItemHash" doesn't conform to snake_case naming style (invalid-name)
ecommerce_cart_system.py:52:0: C0116: Missing function or method docstring (missing-function-docstring)
ecommerce_cart_system.py:52:0: C0103: Function name "generateCheckoutPin" doesn't conform to snake_case naming style (invalid-name)
ecommerce_cart_system.py:56:0: C0116: Missing function or method docstring (missing-function-docstring)
ecommerce_cart_system.py:56:0: C0103: Function name "exportCartReceipt" doesn't conform to snake_case naming style (invalid-name)
ecommerce_cart_system.py:57:8: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
ecommerce_cart_system.py:57:8: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
ecommerce_cart_system.py:63:0: C0116: Missing function or method docstring (missing-function-docstring)
ecommerce_cart_system.py:63:0: C0103: Function name "getCartSummary" doesn't conform to snake_case naming style (invalid-name)
ecommerce_cart_system.py:63:19: W0621: Redefining name 'cart_items' from outer scope (line 6) (redefined-outer-name)
ecommerce_cart_system.py:65:8: W0612: Unused variable 'k' (unused-variable)
ecommerce_cart_system.py:71:0: C0116: Missing function or method docstring (missing-function-docstring)
ecommerce_cart_system.py:4:0: W0611: Unused import math (unused-import)

-----------------------------------
Your code has been rated at 3.86/10
```

After :

```
(venv) PS C:\Users\Prabhakar\L5_47_ecommerce_cart_system> Get-Content pylint_report.txt

-------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 3.86/10, +6.14)
```

2). bandit_report_before.txt

```
(venv) PS C:\Users\Prabhakar\L5_47_ecommerce_cart_system> Get-Content bandit_report.txt
Run started:2026-10-14 07:40:14.303017+00:00

Test results:
>> Issue: [B110:try_except_pass] Try, Except, Pass detected.
   Severity: Low   Confidence: High
   CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b110_try_except_pass.html
   Location: .\ecommerce_cart_system.py:28:4
27              del cart_items[item_id]
28          except:
29              pass
30

--------------------------------------------------
>> Issue: [B307:blacklist] Use of possibly insecure function - consider using safer ast.literal_eval.
   Severity: Medium   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b307-eval
   Location: .\ecommerce_cart_system.py:44:19
43          base_price = cart_items[item_id]["price"]
44          final_amount = eval(discount_expr)
45          return final_amount
```

```
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
   Severity: High   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
   Location: .\ecommerce_cart_system.py:49:11
48      def generateItemHash(item_id):
49          return hashlib.md5(item_id.encode()).hexdigest()
50

--------------------------------------------------
>> Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes.
   Severity: Low   Confidence: High
   CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random
   Location: .\ecommerce_cart_system.py:53:11
52      def generateCheckoutPin():
53          return random.randint(1000, 9999)
54

--------------------------------------------------
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detected, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b605_start_process_with_a_shell.html
   Location: .\ecommerce_cart_system.py:60:4
59          f.close()
60          os.system("echo Receipt saved to: " + filename)
61

--------------------------------------------------

Code scanned:
        Total lines of code: 62
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 2
                Medium: 1
                High: 2
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 5
Files skipped (0):
```

After :

```
(venv) PS C:\Users\Prabhakar\L5_47_ecommerce_cart_system> Get-Content bandit_report.txt
Run started:2026-10-14 07:50:59.657860+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 121
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

3). flake8_report_before.txt

```
(venv) PS C:\Users\Prabhakar\L5_47_ecommerce_cart_system> Get-Content flake8_report.txt
ecommerce_cart_system.py:4:1: F401 'math' imported but unused
ecommerce_cart_system.py:28:5: E722 do not use bare 'except'
ecommerce_cart_system.py:33:5: F841 local variable 'handling_fee' is assigned to but never used
ecommerce_cart_system.py:34:32: E711 comparison to None should be 'if cond is None:'
ecommerce_cart_system.py:43:5: F841 local variable 'base_price' is assigned to but never used
ecommerce_cart_system.py:76:80: E501 line too long (86 > 79 characters)
ecommerce_cart_system.py:84:7: W292 no newline at end of file
```

After :

```
(venv) PS C:\Users\Prabhakar\L5_47_ecommerce_cart_system> Get-Content flake8_report.txt
(venv) PS C:\Users\Prabhakar\L5_47_ecommerce_cart_system> python -m py_compile ecommerce_cart_system.py
```

Corrected Python program :

ecommerce_cart_system.py > ...

```python
import hashlib
import json
import logging
import math
import secrets
from pathlib import Path

LOGGER = logging.getLogger(__name__)

CART_ITEMS = {
    "ITEM1": {"name": "Wireless Mouse", "price": 25.0, "qty": 2},
    "ITEM2": {"name": "Mechanical Keyboard", "price": 80.0, "qty": 1},
    "ITEM3": {"name": "USB-C Hub", "price": 45.0, "qty": 3},
}


def add_item_to_cart(item_id, name, price, qty=1, audit_log=None):
    """Add a new item to the cart after validating its details.

    Return False if the item ID already exists. Invalid item details raise
    ValueError rather than being added to the cart.
    """
    if not isinstance(item_id, str) or not item_id.strip():
        raise ValueError("Item ID must be a non-empty string.")
    if not isinstance(name, str) or not name.strip():
        raise ValueError("Item name must be a non-empty string.")
    if isinstance(price, bool) or not isinstance(price, (int, float)):
        raise ValueError("Price must be a number.")
    if not math.isfinite(price) or price < 0:
        raise ValueError("Price must be finite and cannot be negative.")
    if isinstance(qty, bool) or not isinstance(qty, int) or qty <= 0:
        raise ValueError("Quantity must be a positive integer.")
    if item_id in CART_ITEMS:
        LOGGER.warning("Item %s already exists in the cart.", item_id)
        return False

    CART_ITEMS[item_id] = {
        "name": name.strip(),
        "price": float(price),
        "qty": qty,
    }

    if audit_log is not None:
        audit_log.append(f"Added {item_id} at {float(price):.2f} each.")
    LOGGER.info("Added item %s to the cart.", item_id)
```

ecommerce_cart_system.py > ...

```python
def remove_item_from_cart(item_id):
    """Remove an item from the cart and report whether it was present."""
    removed_item = CART_ITEMS.pop(item_id, None)
    if removed_item is None:
        LOGGER.warning("Cannot remove %s: item was not found.", item_id)
        return False

    LOGGER.info("Removed item %s from the cart.", item_id)
    return True


def check_cart_status(item_id):
    """Return True when the item exists and has a positive quantity."""
    item = CART_ITEMS.get(item_id)
    return item is not None and item["qty"] > 0


def calculate_discounted_total(item_id, discount_rate=0.10):
    """Return the discounted unit price for an item.

    ``discount_rate`` is a fraction from 0.0 to 1.0. Accepting a numeric rate
    instead of an expression string avoids executing user-supplied code.
    """
    item = CART_ITEMS.get(item_id)
    if item is None:
        raise KeyError(f"Item {item_id!r} was not found in the cart.")
    is_numeric_rate = isinstance(discount_rate, (int, float))
    if isinstance(discount_rate, bool) or not is_numeric_rate:
        raise ValueError("Discount rate must be a number between 0 and 1.")
    if not math.isfinite(discount_rate) or not 0 <= discount_rate <= 1:
        raise ValueError("Discount rate must be between 0 and 1.")

    return round(item["price"] * (1.0 - discount_rate), 2)


def generate_item_hash(item_id):
    """Return a SHA-256 digest for a product identifier.

    This digest is an identifier/checksum, not an authentication mechanism.
    """
    if not isinstance(item_id, str):
        raise TypeError("Item ID must be a string.")
    return hashlib.sha256(item_id.encode("utf-8")).hexdigest()
```

```python
def generate_checkout_pin():
    """Generate a four-digit PIN using a cryptographically secure generator."""
    return secrets.randbelow(9000) + 1000


def export_cart_receipt(filename="receipt.txt"):
    """Write a JSON-formatted snapshot of the current cart to a file."""
    receipt_path = Path(filename)
    receipt_path.write_text(
        json.dumps(CART_ITEMS, indent=2, sort_keys=True),
        encoding="utf-8",
    )
    LOGGER.info("Receipt saved to %s", receipt_path)
    return receipt_path


def calculate_cart_total(items=None):
    """Calculate the total value of the selected cart items."""
    source_items = CART_ITEMS if items is None else items
    total = sum(
        float(details["price"]) * int(details["qty"])
        for details in source_items.values()
    )
    return round(total, 2)


def get_cart_summary(items=None):
    """Return the total quantity of items with a positive quantity."""
    source_items = CART_ITEMS if items is None else items
    return sum(
        int(details["qty"])
        for details in source_items.values()
        if int(details["qty"]) > 0
    )


def main():
    """Run a short demonstration of the cart operations."""
    logging.basicConfig(
        level=logging.INFO,
        format="%(levelname)s: %(message)s",
    )
    audit_log = []

    add_item_to_cart("ITEM4", "Monitor Stand", 35.0, 1, audit_log)
    remove_item_from_cart("ITEM2")

    print("Is ITEM1 in cart:", check_cart_status("ITEM1"))
    print("Discounted Price:", calculate_discounted_total("ITEM1"))
    print("Item Hash:", generate_item_hash("ITEM1"))
    print("Checkout PIN generated securely.")
    print("Total items:", get_cart_summary())
    print("Cart total:", f"${calculate_cart_total():.2f}")

```

Identified Issues and Fixes : issues.md

Sl. No.	Issue	Type / Severity	Original Line(s)	Description	Fix Approach
1	Dynamic eval() execution	Security — High	43	Evaluating an expression string can execute arbitrary Python code if the value comes from untrusted input.	Replaced expression evaluation with a numeric discount rate, validated the input, and calculated the discounted price directly.
2	Shell command built using input	Security — High	59	Using os.system() with a filename concatenated into a command can introduce shell injection vulnerabilities.	Removed the shell command and used pathlib.Path.write_text() to write the receipt directly.
3	Hard-coded payment secret	Security — High	71, 80	A secret-like payment gateway key was embedded in the source code and printed, risking exposure through source control or logs.	Removed the hard-coded key and the statement that printed it. Real credentials should be stored securely and never printed.
4	Predictable PIN generator	Security — Medium	52	The random module is not suitable for generating security-sensitive tokens such as checkout PINs.	Replaced it with secrets.randbelow() to generate a four-digit PIN securely.
5	MD5 digest	Security / Reliability — Medium	48	MD5 is outdated for applications requiring collision-resistant hashing.	Replaced MD5 with SHA-256 and clarified that the digest is used as an identifier or checksum, not for authentication.
6	Mutable default argument	Bug / Maintainability — Medium	14	Using audit_log=[] as a default argument can cause the same list to be shared across multiple function calls.	Changed the default argument to None and handled the log list inside the function.
7	Bare exception handling	Reliability — Medium	27–28	A bare except catches unrelated errors, while pass silently hides failures.	Replaced the exception-handling approach with dict.pop(item_id, None) and added clear success or failure handling.
8	Missing input validation	Reliability — Medium	14–20, 41–44	Invalid product IDs, empty names, negative prices, non-positive quantities, or invalid discounts could cause incorrect cart operations.	Added input validation and raised appropriate ValueError or TypeError exceptions for invalid values.

Sl. No.	Issue	Type / Severity	Original Line(s)	Description	Fix Approach
9	File handling without a context manager	Maintainability — Low	56–58	Manually opening and closing files can leave resources unmanaged if an error occurs.	Used Path.write_text() for simpler and safer receipt-writing operations.
10	Incorrect comparison with None and unused variable	Style / Maintainability — Low	32–33	Comparing a value with None using == is non-idiomatic, and an unused variable reduces code clarity.	Used an explicit is not None check and removed the unused variable.
11	Poor naming and formatting	Style — Low	14, 20; import at 4	Inconsistent function naming, unused imports, and older string-formatting styles reduce readability and maintainability.	Adopted snake_case naming, removed unused imports, and used f-strings and consistent formatting.
12	Fragile cart total tracking	Correctness — Medium	19, 24–28	Maintaining a separate global total can produce incorrect results if an item is removed without updating the total.	Removed the stale global counter and calculated the current total directly from the cart contents.

Reflection.md

Lab 5 — Reflection

1. What is static code analysis, and how is it different from testing by running a program?

Static code analysis examines source code without executing it. Pylint checks code quality and common programming mistakes, Flake8 checks style and formatting rules, and Bandit looks for common security weaknesses. Running tests is still important because tests reveal behaviour for selected inputs, while static analysis can flag risky patterns even when a test has not triggered them. Neither approach replaces the other.

2. What kinds of problems did the tools help identify?

The analysis focused on a dangerous `eval()` call, shell-command construction, a hard-coded secret-like value, a predictable random-number generator, MD5 hashing, a mutable default argument, broad exception handling, and style problems. Some correctness and security concerns also required reading the code rather than relying only on tool output. For example, removing an item did not update the separate global total, and the program printed the hard-coded payment key.

3. Which fixes were most important, and why?

The most important fixes were removing `eval()`, removing the shell command built from a filename, and deleting the hard-coded payment secret. These reduce the risk of code execution, shell injection, and credential exposure. Using `secrets` for a PIN and SHA-256 for the item digest improves the security posture. Input validation and clearer error handling also make the cart more reliable for unexpected values.

4. How would static analysis fit into a future development workflow?

I would run Pylint, Bandit, and Flake8 after making changes and before committing code. I would review high-severity security findings first, fix correctness and reliability problems next, and then address style warnings. I would also keep tests for key cart operations, such as adding and removing products, validating quantities, calculating discounts, and exporting receipts. Running the checks automatically in a CI workflow would help prevent similar problems from returning.

Conclusion :

In this experiment, static code analysis was performed on an e-commerce cart system using Pylint, Bandit, and Flake8. The identified issues were addressed, and the corrected program was executed successfully. The final Pylint score was 10.00/10, Bandit reported no issues, and Flake8 produced no reported style violations. This experiment demonstrated how static analysis tools can help improve Python code quality, security, and maintainability.